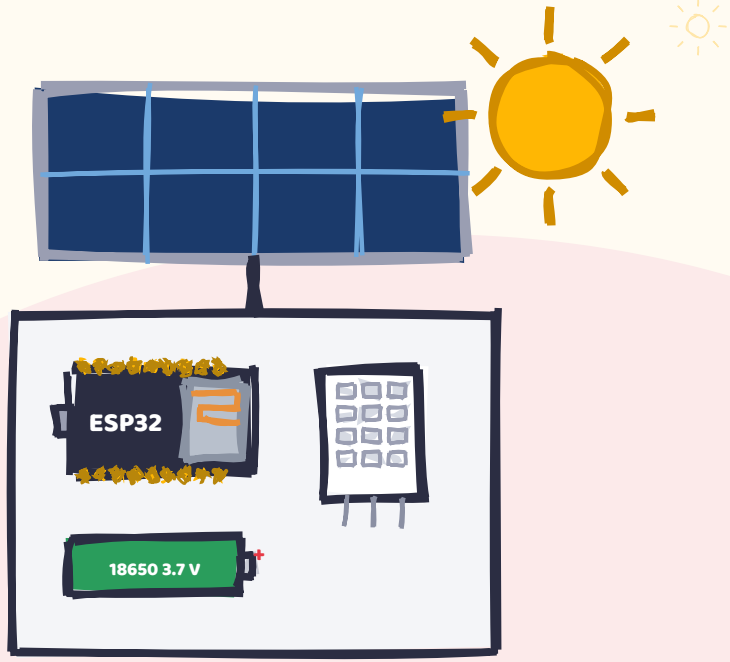
Tech Leaders · Class 1

Solar-Powered Off-Grid Weather Node

A sensor that runs for months with no power socket

Solar + Li-ion | ESP32 deep sleep | Energy budgeting



Did you know?
An ESP32 chip in deep sleep can use about 10 microamps: around 10,000 times less than when Wi-Fi is on.

LEARN

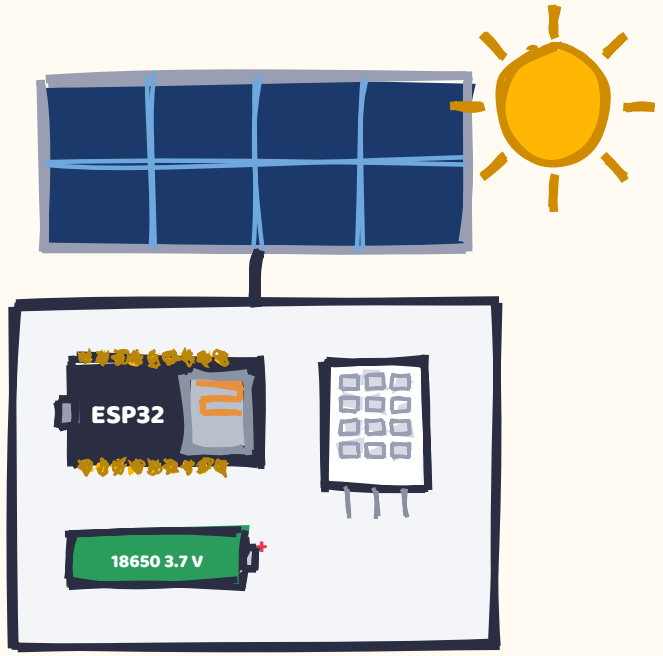
Engineering challenge

No socket, no cable, no visits for months

The problem

Glacier lakes in the north, desert farms in Thar, remote roads: places that need weather data but have **no electricity**. Changing batteries every week is impossible.

Your job: design a node that **powers itself** and keeps reporting.



Bigger battery?

Run a long cable?

Solar + smart sleeping!

Today: you build a solar-charged ESP32 weather node that wakes, measures, sends data over Wi-Fi and sleeps, then you calculate how long it can survive.

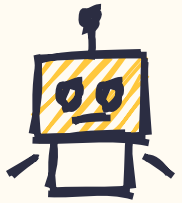
Learning aims

Today I will learn to...

1. Wire a **solar panel, TP4056 charger and 18650 cell** safely
2. Program an ESP32 **wake → read → send → deep sleep** cycle
3. Send readings to a **cloud dashboard** (ThingSpeak)
4. **Measure** awake and sleep current with a multimeter
5. Calculate an **energy budget** and battery life, then improve it

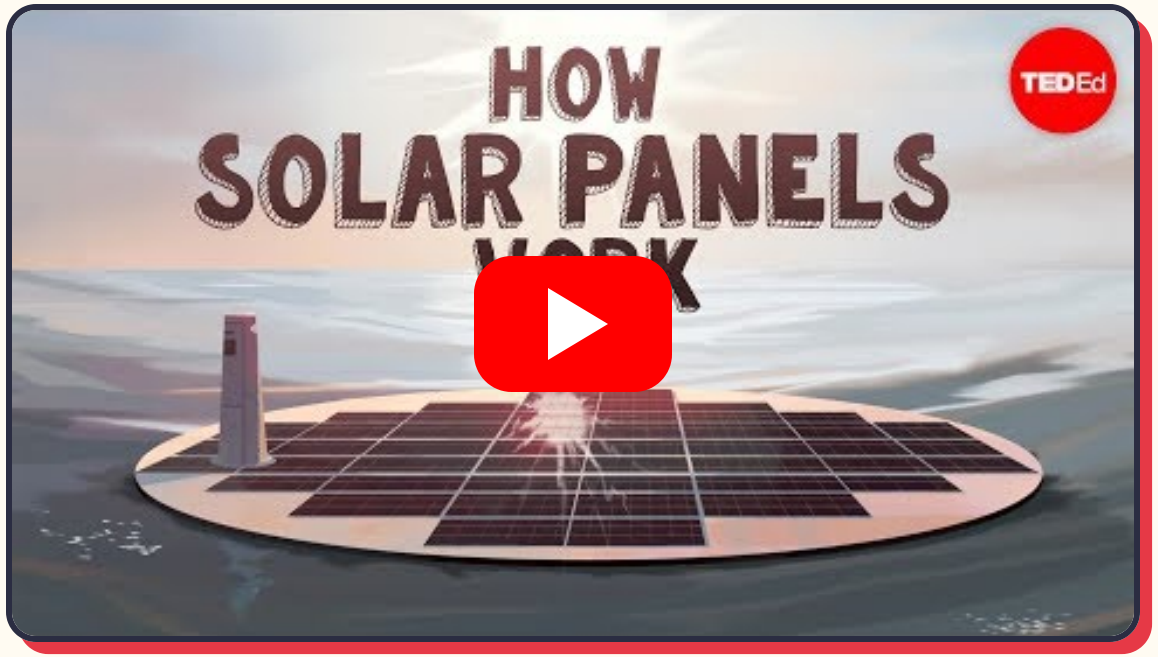
Keywords

energy budget · mAh · deep sleep · duty cycle · TP4056 · voltage divider · IoT



Watch and think

Video time!



How do solar panels work?

TED-Ed (Richard Komp) · about 5 minutes · youtu.be/xKxrkht7CpY

Watch for...

1. What happens to **electrons** when sunlight hits silicon?
2. Why are panels made from two layers of silicon?
3. Why is a solar cell's efficiency limited?

Scan to watch

After the video: Our node must run all night on its battery. Which way should the panel face in Islamabad to charge it fastest, and why?

Real world

Who needs off-grid sensors?



Glacier lake warnings
Remote stations in the mountains watch for floods from melting glaciers



Smart farms
Soil and weather sensors spread across huge fields



Wildlife trackers
Solar collars report animal positions for years



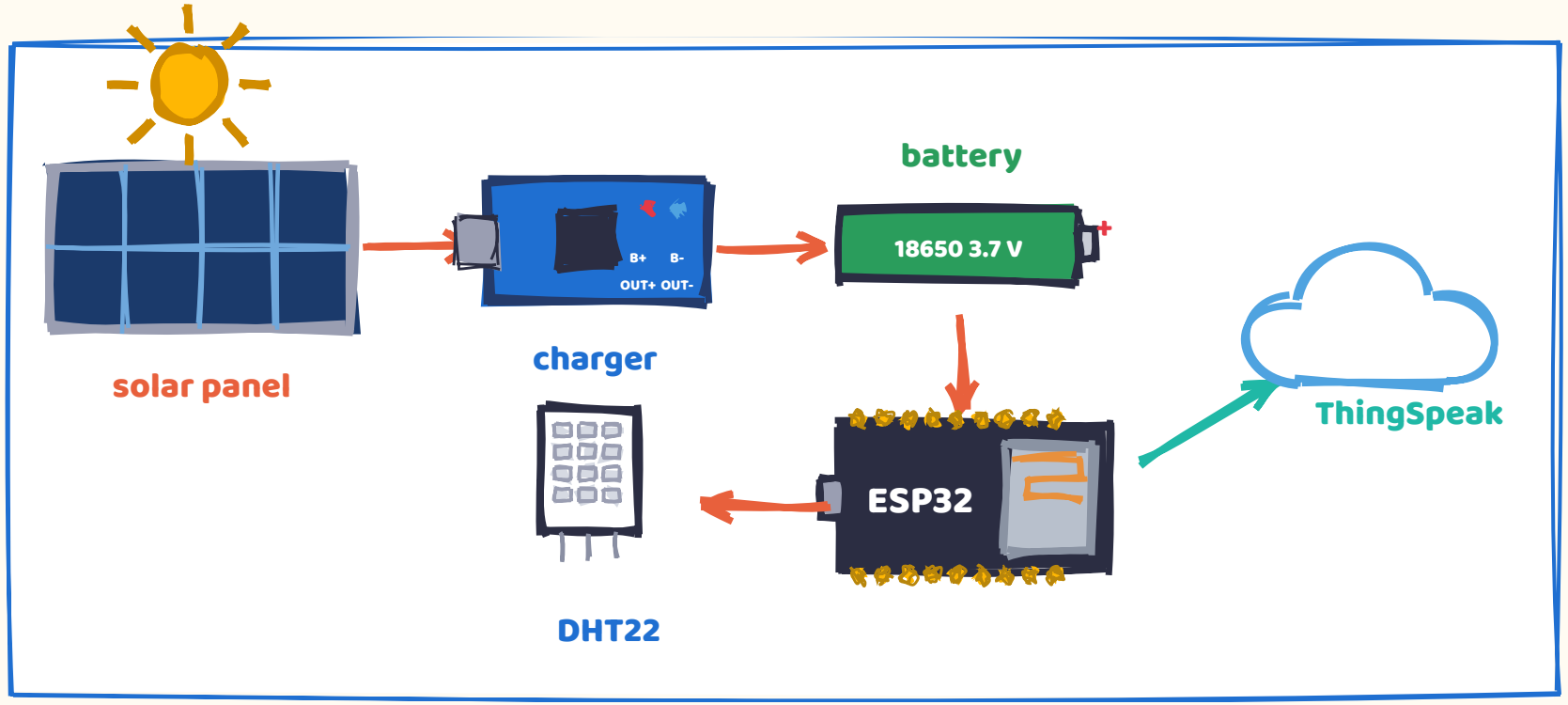
Space probes
Every watt is budgeted, just like our node

? What would happen to a remote sensor that wakes every second instead of every 10 minutes?

Key idea
Most of an IoT device's life is spent asleep. Sleep design decides battery life.

System design

How energy and data flow



Energy path

Sun → panel → **TP4056** charges the **18650** → battery powers the ESP32.

Data path

DHT22 → ESP32 → Wi-Fi → ThingSpeak chart you can open anywhere.

Two budgets

Engineers design the ENERGY budget and the DATA budget together.

Physics refresher

Power, capacity and average current

Quantity	Formula	Unit	Example
Power	P = V × I	watts (W)	5 V × 0.1 A = 0.5 W
Charge stored	capacity	mAh	2000 mAh = 2000 mA for 1 hour
Battery life	capacity ÷ average current	hours	2000 ÷ 10 mA = 200 h
Energy	E = P × t	Wh	0.5 W × 3 h = 1.5 Wh

Average current is the key number

A node that draws 120 mA for 6 s and almost nothing for the rest of 10 minutes has a **tiny average**. That is the whole trick of IoT design.

18650 facts

Nominal **3.7 V**, full at **4.2 V**, treat **3.0 V** as empty. Typical capacity 2000-3000 mAh (check the label, fakes exaggerate).

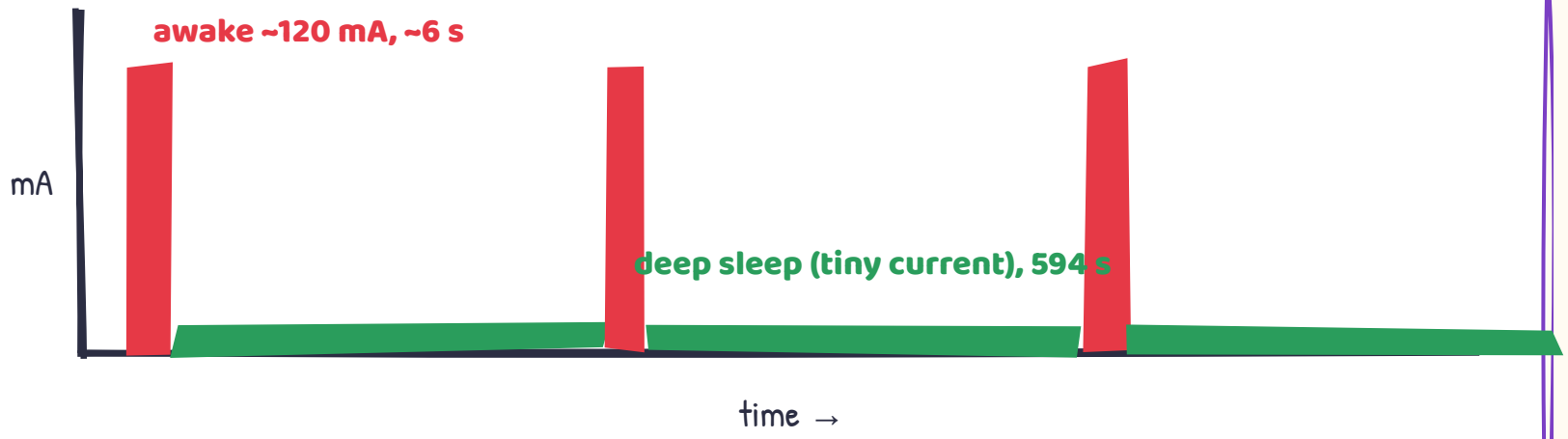
Check your units!

mA vs µA (1 mA = 1000 µA) is the most common mistake.

LEARN

Big idea

Duty cycling: wake, work, sleep



Duty cycle

Awake 6 s every 600 s = **1%** duty cycle. The other 99% decides battery life.

Deep sleep

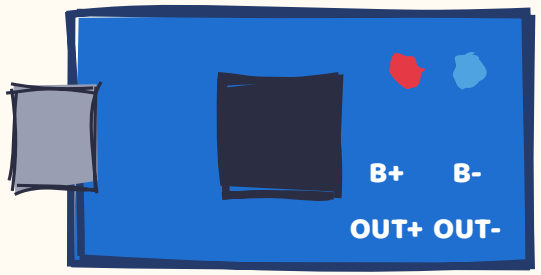
The ESP32 switches off its CPU and Wi-Fi. Only the RTC timer stays awake to wake it later. When it wakes, the program starts again from **setup()**.

Gotcha

loop() never runs in this project. Everything happens in setup(), then sleep.

Meet the parts

Charger, cell and board



Use the protected version

Buy the TP4056 board with **6 pads** (IN, B, OUT). The OUT pads pass through a protection chip that cuts off at about **2.5 V** and on short circuits.

ESP32 pin	Use
GPIO 4	DHT22 data
GPIO 34	battery voltage (ADC, input only)
VIN / 5V	power in
3V3	3.3 V out (sensors)

TP4056 pad	Connects to
IN+ / IN-	from the solar panel (or USB)
B+ / B-	to the 18650 cell
OUT+ / OUT-	to the load (ESP32), protected
Red / blue LED	charging / full

Solar input

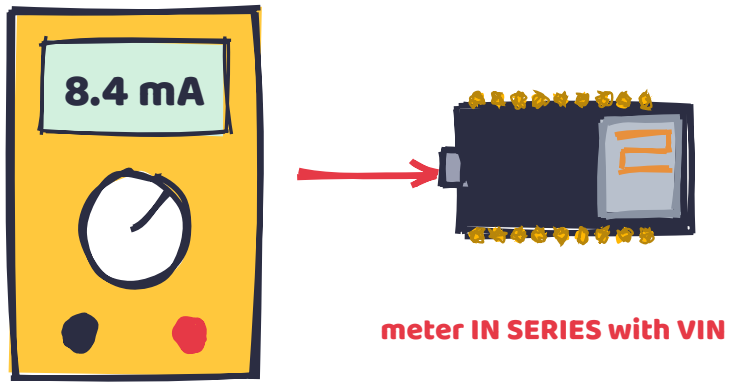
A 5-6 V, 1 W panel gives roughly 150-200 mA in full sun. The TP4056 accepts 4.5-5.5 V; a 6 V panel normally sags into range under load.

ADC1 only!
When Wi-Fi is on, the ADC2 pins stop working. GPIO 34 is on ADC1, so it is safe.

LEARN

Measure like an engineer

How much current does the node really use?



meter IN SERIES with VIN

State	Meter setting	Result
Awake (Wi-Fi on)	10 A socket	____ mA
Deep sleep	mA socket, 200 mA range	____ mA
Awake time per cycle	stopwatch / Serial	____ s

How to measure

Break the **OUT+ → VIN** wire and put the meter **in series**: red probe to OUT+, black probe to VIN. Choose the right socket first!

Surprise coming!

On a DevKit board, deep sleep may still show **several mA**. The chip sleeps, but the USB chip, power LED and regulator do not. That is the real lesson of this class.

Protect the meter

Wi-Fi bursts can pass 200 mA and blow the mA fuse. Use the 10 A socket for awake.

LEARN

Maths

The energy budget, worked example

Average current = $(I_{awake} \times t_{awake} + I_{sleep} \times t_{sleep}) \div T$

= (120 mA × 6 s + 8 mA × 594 s) ÷ 600 s

= (720 + 4752) ÷ 600 = **9.1 mA**

Battery life = 2000 mAh ÷ 9.1 mA ≈ **220 h ≈ 9 days** with no sun

Can the sun keep up?

Need: 9.1 mA × 24 h ≈ **220 mAh/day**. Panel: ~150 mA × ~5 sunny hours ≈ **750 mAh/day** before losses. Sunny days: yes. A cloudy week: the battery must cover it.

Same node, bare ESP32 module (0.15 mA sleep)

(720 + 0.15 × 594) ÷ 600 ≈ **1.35 mA** → about **62 days**. The board design matters more than the code!

Use YOUR numbers

Replace 120, 8 and 6 with what you measured.

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Lithium cells store a lot of energy
2. Kit per group
3. Test the solar charger
4. Add the battery
5. Power the ESP32 from the battery
6. Add the sensor and battery monitor
7. Create the cloud dashboard

Safety first

Lithium cells store a lot of energy

- **Never short** the + and - of an 18650. It can get very hot or catch fire.
- **Protected cells only** and a protected TP4056 board. Check polarity before inserting.
- **Never charge unattended** and never charge a swollen, dented or wet cell.
- **No metal on the bench** loose wires, rings or tools can bridge terminals.
- **Store at about 3.7 V** in a fireproof box. Teacher keeps the cells between classes.



Teacher rule

Students wire with the cell OUT of the holder. The teacher checks, then inserts it.

Build it

Kit per group

ESP32 DevKit
+ micro-USB cable

Solar panel 5-6 V 1 W
the power source

18650 + holder
protected cell

TP4056 (protected)
charger board

DHT22 sensor
temperature + humidity

Digital multimeter
current + voltage

Also: 2 × 10k resistors (battery divider), breadboard, M-M and M-F jumpers, laptop with Arduino IDE 2 + ESP32 board package + DHT library, desk lamp for indoor solar tests.

Indoor sun
A strong desk lamp close to the panel works when the sky is cloudy.

Build step 1 of 6

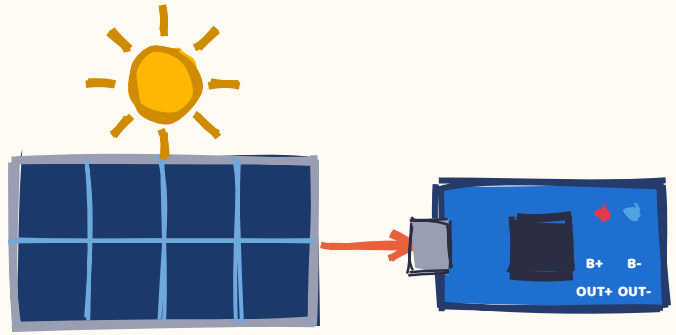
Test the solar charger

a Set the multimeter to **DC volts (20 V)**. Measure the panel's open-circuit voltage in sun or under a lamp. Record it.

b Connect panel **red** → **IN+** and **black** → **IN-** on the TP4056. No battery yet.

c Shine light on the panel. The TP4056 LEDs should light (red or blue flashing with no cell is normal).

d Cover the panel with your hand. The LEDs go out. Light is now your power switch!



TP4056 charger

Check: panel voltage is about 5-7 V in light and the charger LEDs react to shade.

Never connect the panel to the ESP32 directly. Always go through the charger and battery.

Build step 2 of 6

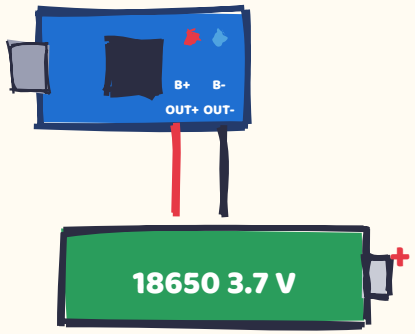
Add the battery

a Teacher checks the cell: voltage between **3.0 and 4.2 V**, no damage.

b Wire the holder **+** → **B+** and **-** → **B-**. Double-check colours and polarity.

c Teacher inserts the cell. Light the panel: the **red LED = charging**, **blue = full**.

d Measure the battery voltage every 10 minutes while charging. Does it rise?



Li-ion cell (protected)

Check: B+ and B- show the same voltage as the cell, and charging stops at about 4.2 V.

Wrong polarity on B+/B- can destroy the board and heat the cell.

MAKE

Build step 3 of 6

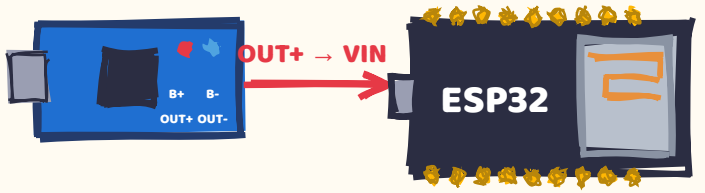
Power the ESP32 from the battery

a Connect **OUT+ → ESP32 VIN** (the 5V pin) and **OUT- → GND**.

b Unplug USB first: never power from USB and battery at the same time on VIN.

c Watch the ESP32 power LED. It should light from the battery alone.

d **Note:** the DevKit's regulator needs about 3.6 V or more. Below that, Wi-Fi bursts can reset the board (a **brownout**).



Check: the ESP32 boots on battery power and runs a blink test.

Upgrade: an HT7333 or MCP1700 low-dropout regulator to the 3V3 pin works down to about 3.4 V and wastes far less power.

Build step 4 of 6

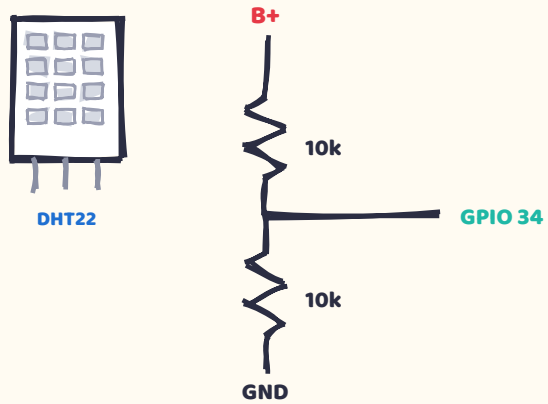
Add the sensor and battery monitor

a DHT22: **+** → **3V3**, **-** → **GND**, **data** → **GPIO 4** (most DHT22 modules already include the 10k pull-up).

b Battery divider: **B+** → **10k** → **GPIO 34** → **10k** → **GND**. GPIO 34 sees half the battery voltage.

c Why halve it? The ESP32 ADC can only measure up to about 3.3 V, and a full cell is 4.2 V.

d Tidy and label the wires. A messy node fails in the field!



Check: a multimeter on GPIO 34 reads about half the battery voltage.

The divider always draws about 0.2 mA. Using 100k resistors cuts that 10 times.

Build step 5 of 6

Create the cloud dashboard

a Teacher creates a free **ThingSpeak** (thingspeak.com) account and one channel per group.

b Add **Field 1** temperature, **Field 2** humidity, **Field 3** battery volts, **Field 4** boot count.

c Copy the channel's **Write API Key** into your code. Keep it private: anyone with it can post to your channel.

d Test in a browser: **api.thingspeak.com/update?api_key=KEY&field1=25** should return a number, not 0.



ThingSpeak channel

Check: the browser test makes a new point appear on Field 1.

The free plan accepts an update about every 15 seconds. Our node sends far less often.

Code it: part 1

Wake, read and decide

```
#include <WiFi.h>
#include <HTTPClient.h>
#include "DHT.h"

const char* SSID = "SchoolWiFi";
const char* PASS = "********";
String KEY = "YOUR_WRITE_KEY";
#define SLEEP_SECONDS 600            // 10 minutes (use 60 to test)

DHT dht(4, DHT22);
RTC_DATA_ATTR int bootCount = 0;     // survives deep sleep

void setup() {
  Serial.begin(115200);
  bootCount++;
  dht.begin();
  delay(2000);                       // DHT22 needs ~2 s
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  float vbat = analogReadMilliVolts(34) * 2 / 1000.0;
  sendData(t, h, vbat);
  goToSleep();
}
void loop() { }                      // never reached
```

Everything in setup()

After deep sleep the ESP32 restarts from the top, so the whole job lives in **setup()**.

RTC_DATA_ATTR

Normal variables are wiped by deep sleep. This one is kept in RTC memory, so **bootCount** keeps counting.

analogReadMilliVolts

Uses the ESP32's built-in calibration. × 2 undoes the divider.

Code it: part 2

Send, then sleep

```
void sendData(float t, float h, float v) {
  WiFi.begin(SSID, PASS);
  unsigned long start = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - start < 10000)
    delay(100);                          // give up after 10 s
  if (WiFi.status() == WL_CONNECTED && !isnan(t)) {
    HTTPClient http;
    String url = "http://api.thingspeak.com/update?api_key=" + KEY +
                 "&field1=" + String(t, 1) + "&field2=" + String(h, 0) +
                 "&field3=" + String(v, 2) + "&field4=" + String(bootCount);
    http.begin(url);
    Serial.println(http.GET());          // 200 = OK
    http.end();
  }
}

void goToSleep() {
  WiFi.disconnect(true);
  WiFi.mode(WIFI_OFF);
  esp_sleep_enable_timer_wakeup((uint64_t)SLEEP_SECONDS * 1000000ULL);
  esp_deep_sleep_start();                // sleep now
}
```

Always time out

If Wi-Fi is down, a node that keeps trying will **drain its battery**. 10 s, then sleep anyway.

isnan(t)

A failed DHT read returns NaN (not a number). Never upload bad data.

Microseconds

The timer counts in µs: 600 s × 1,000,000. The **ULL** stops the number overflowing.

MAKE

Build step 6 of 6

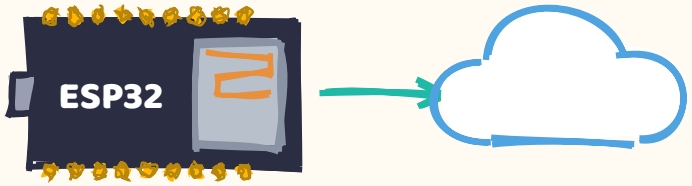
Upload and test the cycle

a Set **SLEEP_SECONDS = 60** for testing. Upload over USB, then open the Serial Monitor at 115200.

b You should see the HTTP code **200**, then nothing for 60 s (it is asleep), then it repeats.

c Open your ThingSpeak channel: a new point every minute, boot count going up 1, 2, 3…

d Unplug USB and run on the battery. Points must keep arriving. Then set the interval back to 600.



200 OK

Check: 4 points in a row arrive on battery power alone.

To re-upload while it sleeps, hold the BOOT button as the upload starts.

Experiment & optimise

How long can you make it last?

Wake interval	Average current	Battery life (no sun)
60 s	____ mA	____ days
300 s	____ mA	____ days
600 s	____ mA	____ days
1800 s	____ mA	____ days

Trade-off

Longer sleep = longer life, but less frequent data. What does YOUR user need?

Optimisation ideas

- [ ] Static IP: connects faster, less awake time
- [ ] Power the DHT22 from a GPIO only while reading
- [ ] 100k divider resistors
- [ ] LDO regulator instead of the DevKit regulator
- [ ] Send only if the value changed a lot
- [ ] Bare ESP32 module instead of a DevKit

Test like an engineer

Troubleshooting

Symptom	Likely cause / fix
Resets when Wi-Fi starts	battery low or weak regulator: charge, add LDO or 470 µF capacitor
HTTP code -1	no Wi-Fi or wrong SSID/password
ThingSpeak shows 0	wrong Write API key or updating faster than 15 s
Temperature = nan	DHT22 wiring, or not waiting 2 s
Battery reads 0 or 6 V	divider wiring, or wrong pin (use GPIO 34)

Debug checklist

Use Serial prints at each stage: woke, read, connected, sent, sleeping. Test on USB power first, battery second.

Keep a log: date, battery volts, sun hours. Real engineers prove their designs with data.

Level up!

Level up

Choose your challenge

Low-battery mode: below 3.5 V, wake every hour instead of every 10 minutes.

ESP-NOW: send to a nearby gateway without joining Wi-Fi: much shorter awake time.

Enclosure: 3D-print a vented, rain-shielded case (a Stevenson screen) for outdoor use.

Sun tracker: log panel voltage too and plot energy in vs energy out over a week.

Think like an engineer

A real station must survive a week of winter fog. How big must the battery and panel be? Show your calculation.

Boss level

Design for 7 days with no sun: 220 mAh/day × 7 ≈ 1540 mAh, plus a safety margin.

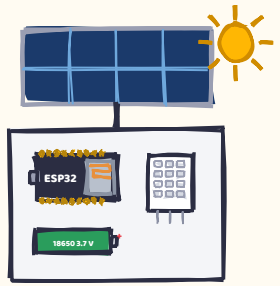
Show what you know

What did we learn?

- [x] Solar → TP4056 → 18650 → ESP32 is the energy chain
- [x] Deep sleep and short awake time set battery life
- [x] Average current = weighted mix of awake and sleep
- [x] Always time out Wi-Fi and protect lithium cells
- [x] Measure, calculate, optimise: that is engineering

Exit ticket: a node uses 100 mA for 5 s and 1 mA for 595 s. What is its average current?

At home
Read the label on a power bank. How many days could it run your node?



Next class:
Autonomous Maze Solving Robot (PID)!